



Informatikos inžinerijos studijų programa

Inžinerinis projektas

*T120B204 Sistemų integracijos technologijos
ir saugių paslaugų kūrimas*

REST API technologijas

ATLIKO:

Gediminas Viliūnas
(Vardas Pavardė)

(Parašas)

IFB-2
(Grupė)

DĖSTYTOJAS:

(Vardas Pavardė)

(Parašas)

DARBAS ATIDUOTAS:

___ d. _____ mėn. 2022

Contents

Užduotis	2
Trumpas sistemos aprašymas	3
Architektūros aprašymas	4
2.1 Aukšto lygio architektūros komponentai	4
2.2. Išorinių paslaugų integracija (Third-party APIs)	5
2.3. Saugumo sprendimai	5
Sistemos projektavimas (UML Diagramos).....	5
3.1. Esysių diagrama.....	5
3.2. Diegimo diagrama (Deployment Diagram).....	6
programinis kodas.....	7
Rezultatų apibendrinimas	7

Užduotis

Darbo tikslas:

Suprojektuoti ir realizuoti investicijų simuliacinio informacinę sistemą „RentVest“, praktiškai pritaikant REST API (angl. Representational State Transfer) architektūrinio stiliaus principus bei pademonstruoti gilus šios technologijos veikimo, architektūrinių savybių bei saugumo mechanizmų suvokimą.

Keliami uždaviniai:

Siekiant gauti maksimalų įvertinimą (pagal 2-ąjį užduoties tipą), darbe turi būti įgyvendinti šie techniniai ir inžineriniai uždaviniai:

- Sistemos architektūros projektavimas:
 - Sukurti sluoksniuotą (Layered) sistemą, atskiriant klientinę dalį (Client) nuo serverinės dalies (Server).
 - Įdiegti tarpinį API Gateway sluoksnį, užtikrinantį vieningą įėjimo tašką ir paslėpiančią vidinę sistemos struktūrą.
 - Užtikrinti „Stateless“ (būsena nesaugoma) veikimo principą serverinėje dalyje.
- Funkcionalumo realizavimas naudojant REST API:
 - Suprogramuoti serverinę dalį (.NET Web API), kuri teikia resursus (investicijas, vartotojus, naujienas) naudojant standartinius HTTP metodus (GET, POST, DELETE) ir JSON duomenų formatą.
 - Sukurti klientinę vartotojo sąsają (.NET MVC), kuri veikia kaip REST klientas ir komunikuoja su serveriu tik per HTTP užklausas.
- Integracija su išorinėmis sistemomis:

- Sistemos logikoje panaudoti ir apjungti duomenis iš bent 3 skirtingų išorinių REST API:
 - Finansinių rinkų duomenų API (pvz., Alpha Vantage) istorinėms kainoms gauti.
 - Valiutų kursų API (pvz., Frankfurter) konvertavimui.
 - Naujienų API (pvz., NewsAPI) kontekstinei informacijai gauti.
- Realizuoti duomenų podėliavimo (Cacheability) mechanizmą vietinėje duomenų bazėje, siekiant optimizuoti užklausų skaičių ir sistemos greitaveiką.
- Kibernetinio saugumo užtikrinimas:
 - Implementuoti saugų vartotojų autentifikacijos mechanizmą naudojant JWT (JSON Web Tokens) standartą.
 - Realizuoti RBAC (Role-Based Access Control) prieigos valdymą, sukuriant skirtingas teises „Vartotojo“ ir „Administratoriaus“ rolėms.
 - Užtikrinti saugų duomenų perdavimą naudojant HTTPS protokolą.
 - Apsaugoti vartotojų slaptažodžius naudojant kriptografinį maišos (hashing) algoritmą (pvz., BCrypt).
 - Įdiegti apsaugą nuo per didelės apkrovos ir DDoS atakų (Rate Limiting) API Gateway lygmenyje.

Trumpas sistemos aprašymas

Kuriama informacinė sistema „RentVest“ yra investicijų įrankis, skirtas vartotojui pamatyti, kokią finansinę grąžą jis būtų gavęs, jei konkrečią pinigų sumą (pvz., mėnesinę nuomą) būtų investavęs į finansines rinkas pasirinktu istoriniu laikotarpiu.

Sistemos funkcionalumas yra suskirstytas į dvi pagrindines dalis: viešąją ir registruotų vartotojų zoną. Pagrindinės sistemos funkcijos apima:

1. **Vartotojų valdymas:** Sistema užtikrina saugią vartotojų registraciją ir autentifikaciją. Įgyvendintas vaidmenų (angl. *User Roles*) mechanizmas:
 - *Vartotojas:* Gali atlikti simuliacijas, peržiūrėti savo veiksmų istoriją.
 - *Administratorius:* Turi prieigą prie visų sistemos vartotojų sąrašo ir gali stebėti bendrą sistemos naudojamumą.
2. **Investicijų simuliacija:** Vartotojas įveda pradinį duomenį: investuojamą sumą, valiutą (EUR, USD, GBP), investavimo pradžios datą ir rizikos toleranciją (konservatyvi, vidutinė, agresyvi). Sistema, remdamasi šiais duomenimis, suformuoja diversifikuotą portfelį (akcijos, obligacijos, indeksai).
3. **Istorinių duomenų analizė:** Naudojant realius istorinius rinkos duomenis, sistema apskaičiuoja teorinį pelną arba nuostolį per pasirinktą laikotarpį.
4. **Kontekstinė analizė:** Siekiant edukacinio tikslo, sistema ne tik pateikia skaičius, bet ir sugeneruoja (arba suranda) to laikotarpio finansines naujienas, kurios paaiškina rinkos svyravimų priežastis (pvz., kodėl portfelio vertė krito 2023 m. kovą).
5. **Duomenų kaupimas:** Visi simuliacijų rezultatai yra saugomi duomenų bazėje, leidžiant vartotojui analizuoti savo sprendimų istoriją.

Architektūros aprašymas

Projektuojant sistemą pasirinkta **sluoksniuota architektūra** su **API Gateway** modeliu. Toks sprendimas pasirinktas siekiant užtikrinti sistemos, saugumą ir mastelio keitimo galimybes. Sistema remiasi **REST (Representational State Transfer)** architektūriniu stiliumi komunikacijai tarp komponentų.

2.1 Aukšto lygio architektūros komponentai

Sistemą sudaro trys pagrindiniai nepriklausomi komponentai:

- **Klientinė dalis (Frontend - RentVest.Web):**
 - Realizuota naudojant **ASP.NET Core MVC** technologiją.
 - Tai yra vartotojo sąsajos (UI) sluoksnis, atsakingas už duomenų atvaizdavimą ir vartotojo įvesčių validaciją.
 - Šis sluoksnis tiesiogiai nesijungia prie duomenų bazės. Jis veikia kaip REST klientas, siunčiantis HTTP užklausas į „API Gateway“.
 - Naudojama sesija (Session Storage) laikinam JWT žetono (Token) saugojimui vartotojo naršymo metu.
- **Tarpinis sluknis (API Gateway - RentVestGateway):**
 - Realizuotas naudojant **Ocelot** biblioteką .NET aplinkoje.
 - Tai yra vienintelis viešai prieinamas įėjimo taškas (Single Entry Point) į backend sistemą.
 - *Pagrindinės funkcijos:*
 - **Maršrutizavimas (Routing):** Nukreipia užklausas iš viešojo adreso (pvz., /gateway/users) į vidinį mikroserviso adresą (pvz., /api/users). Tai paslepia tikrąją tinklo topologiją nuo kliento.
 - **Saugumas:** Veikia kaip pirminis apsaugos skydas. Čia vyksta pirminė JWT žetono validacija.
 - **Srauto ribojimas (Rate Limiting):** Apsaugo sistemą nuo DDoS atakų, ribojant užklausų skaičių per laiko vienetą (pvz., 5 užklausos per 5 sekundes).
- **Taikomoji programa (Backend API - RentVest.Api):**
 - Realizuota naudojant **ASP.NET Core Web API**.
 - Tai yra sistemos „smegenys“, kur vyksta visa verslo logika.
 - Veikia pagal **REST** principus:
 - **Stateless (Būsena nesaugoma):** Kiekviena HTTP užklausa turi turėti visą reikalingą informaciją (Authorization Header su Bearer Token), kad serveris galėtų ją apdoroti. Serveris nesaugo kliento būsenos tarp užklausų.
 - **Uniform Interface:** Naudojami standartiniai HTTP metodai (GET, POST, DELETE) ir JSON duomenų formatas resursų atvaizdavimui.
 - Šis sluoksnis komunikuoja su duomenų baze per ORM (Entity Framework Core) ir valdo integracijas su trečiųjų šalių API.

2.2. Išorinių paslaugų integracija (Third-party APIs)

Sistema veikia kaip agregatorius, naudojantis tris išorinius REST API:

1. **Alpha Vantage API:** Naudojamas istoriniams akcijų, ETF ir indeksų duomenims gauti.
2. **Frankfurter API:** Naudojamas valiutų konvertavimui (pvz., EUR į USD) pagal istorinius kursus.
3. **NewsAPI.ai:** Naudojamas semantinei naujienų paieškai pagal raktinius žodžius ir datas.

Kadangi išoriniai API turi kvotų ribojimus ir veikia lėčiau nei vietinė DB, sistemoje įgyvendintas **Caching** mechanizmas. Prieš kreipiantis į išorinį API, sistema pirmiausia tikrina vietinę duomenų bazę. Jei duomenys rasti – naudojami jie, jei ne – kreipiamasi į išorinį API ir gautas atsakymas išsaugomas ateičiai.

2.3. Saugumo sprendimai

Sistemoje įgyvendinti šie kibernetinio saugumo sprendimai:

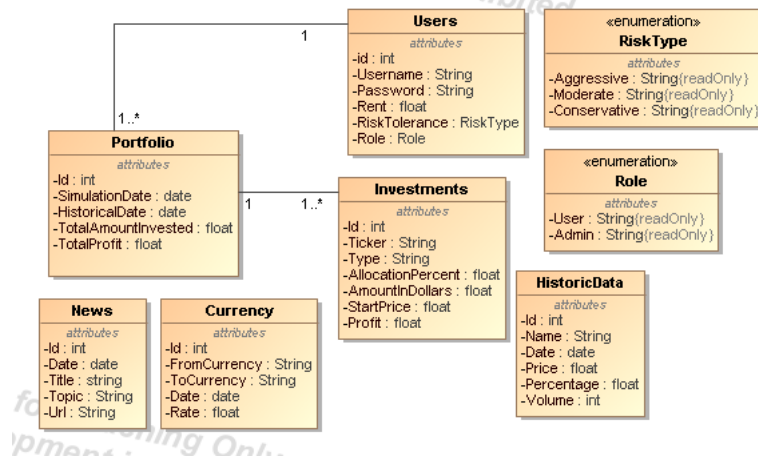
- **Autentifikacija:** Naudojamas **JWT (JSON Web Token)** standartas. Vartotojui prisijungus, išduodamas pasirašytas žetonas, kuriame užkoduotas vartotojo ID ir rolė.
- **Autorizacija:** Naudojamas RBAC (Role-Based Access Control) modelis. Tam tikri *endpoints* (pvz., /api/users/all) yra pasiekiami tik vartotojams, turintiems „Admin“ rolę žetone ([Authorize(Roles = "Admin")]).
- **Slaptažodžių saugumas:** Slaptažodžiai duomenų bazėje niekada nesaugomi atviru tekstu. Naudojamas **BCrypt** algoritmas su druska (salt) slaptažodžių maišai (hashing) sudaryti.
- **Komunikacija:** Visa komunikacija tarp kliento ir serverio priverstinai vyksta per saugų **HTTPS** protokolą

Sistemos projektavimas (UML Diagramos)

Siekiant detaliau pavaizduoti sistemos struktūrą ir komponentų išdėstymą, pateikiamos klasių (Class) ir diegimo diagramos.

3.1. Esybių diagrama

Ši diagrama atvaizduoja pagrindinę (Backend) verslo logiką ir duomenų struktūrą. Ji parodo ryšius tarp duomenų esybių, servisų ir kontrolierių.



1 pav. Esysių diagrama

Pateikta klasių diagrama detalizuoja (serverinės dalies) vidinę struktūrą. Diagramoje išskiriami trys pagrindiniai sluoksniai: duomenų esybės (Entities), verslo logikos servais (Services) ir užklausų valdikliai (Controllers).

Pagrindiniai diagramos elementai:

1. Duomenų modelio esybės (Entities):

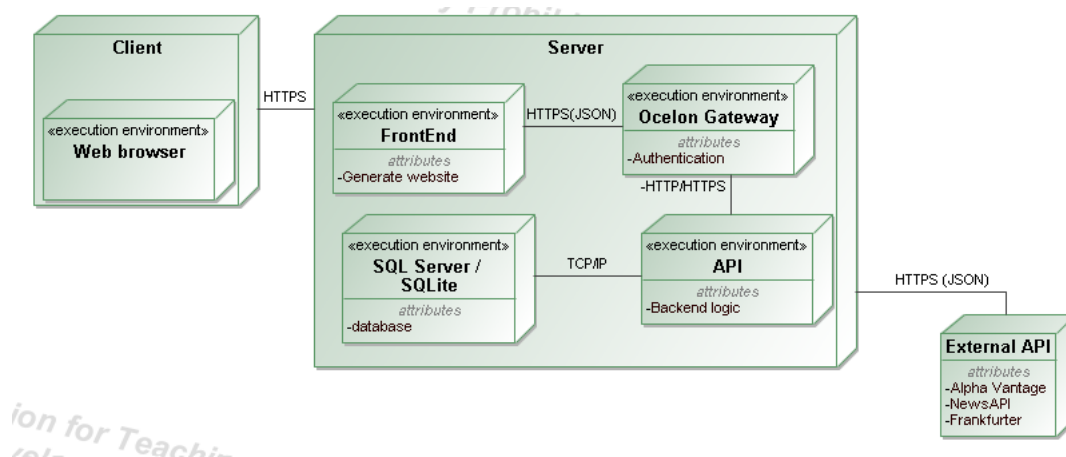
- **Users (Vartotojai):** Tai pagrindinė sistema besinaudojančio asmens klasė. Joje saugomi ne tik autentifikacijos duomenys (Username, HashedPassword), bet ir vartotojo nustatymai, tokie kaip „Rizikos tolerancija“ (RiskTolerance) bei numatytoji nuomos suma (Rent). Šie duomenys naudojami personalizuojant simuliacijas.
- **Portfolio (Portfelis):** Ši klasė reprezentuoja vieną konkrečią, vartotojo atliktą investavimo simuliaciją. Ji veikia kaip istorinis įrašas („snapshot“), kuriame saugoma simuliacijos data, pasirinktas istorinis laikotarpis bei bendras sugeneruotas pelnas (TotalProfit).
- **Investments (Investicijos):** Tai detalizuoti įrašai, priklausantys konkrečiam portfeliui. Kiekvienas įrašas nurodo konkretų finansinį instrumentą (pvz., „IBM“ akcija), jam skirtą pinigų sumą ir gautą pelną.
- **Pagalbinės lentelės (HistoricData, News, Currency):** Šios klasės skirtos saugoti duomenims, gautiems iš išorinių API. Tai realizuoja duomenų podėliavimo (caching) mechanizmą – sistema pirmiausia ieško duomenų čia, o tik jų neradusi kreipiasi į išorinius šaltinius.

Sistemos realizavimo logika:

- **RecommendationController:** Tai yra įėjimo taškas, priimantis HTTP užklausas iš kliento. Jis neįgyvendina sudėtingų skaičiavimų, o deleguoja juos servisams.
- **RecommendationService:** Pagrindinė verslo logikos klasė. Ji atsakinga už investavimo strategijos parinkimą, pelningumo skaičiavimą ir duomenų išsaugojimą duomenų bazėje. Ji komunikuoja su kitais servais, kad surinktų visą reikiamą informaciją.

3.2. Diegimo diagrama (Deployment Diagram)

Ši diagrama parodo fizinę sistemos architektūrą ir tinklo topologiją. Ji iliustruoja, kaip **Ocelot Gateway** veikia kaip tarpininkas tarp vartotojo ir vidinės sistemos.



2pav. Diegimo diagrama

Pateikta diegimo diagrama iliustruoja fizinę sistemos architektūrą ir komponentų sąveiką vykdymo aplinkoje.

- **Kliento lygmuo (Client):** Vartotojas naudoja sistema per interneto naršyklę (*Web Browser*). Komunikacija su serveriu vyksta saugiu **HTTPS** protokolu.
- **Serverio lygmuo (Server):** Visi aplikacijos komponentai talpinami aplikacijų serveryje (kūrimo metu – lokaliame kompiuteryje), tačiau veikia kaip atskiri procesai:
 - **FrontEnd** Atsakingas už vartotojo sąsajos generavimą (MVC) ir užklausų formavimą.
 - **Ocelot Gateway:** Veikia kaip tarpinis sluoksnis (*Reverse Proxy*), nukreipiantis srautą ir užtikrinantis saugumą.
 - **API** Vykdo pagrindinę verslo logiką, skaičiavimus ir duomenų apdorojimą.
- **Duomenų saugojimas:** Duomenys saugomi **SQL Server** (arba **SQLite**) duomenų bazėje, kuri pasiekama per **TCP/IP** protokolą.
- **Išoriniai servisai:** Sistema realiuoju laiku komunikuoja su **External API** (Alpha Vantage, NewsAPI, Frankfurter) per **HTTPS** protokolą **JSON** formatu, siekiant gauti rinkos duomenis ir naujienas.

Programinis kodas

Programinis kodas bus pasiekiamas kaip archyvuotas projektas dėl didelio skaičiaus programinių eilučių

Rezultatų apibendrinimas

1. **Architektūrinis išpildymas:** Sukurta **sluoksniuota (Layered)** sistema, atitinkanti mikroservisų architektūros principus. Sistema suskaidyta į nepriklausomus komponentus: klientinę dalį (**ASP.NET Core MVC**), aplikacijų serverį (**Web API**) ir tarpinį saugumo bei maršrutizavimo sluoksnį (**API Gateway** naudojant **Ocelot**). Tai užtikrina sistemos mastelio keitimą ir komponentų atskiriamumą.
2. **REST API principų taikymas:** Sistema realizuota laikantis **REST** architektūrinio stiliaus. Serveris veikia „**Stateless**“ (būsena nesaugoma) principu, naudodamas standartinius **HTTP** metodus (**GET**, **POST**, **DELETE**) ir **JSON** duomenų formatą. Resursai (Vartotojai, Portfeliai, Naujienos) yra aiškiai apibrėžti ir pasiekiami per unifikuotą sąsają.

3. **Integracija su išorinėmis sistemomis:** Įgyvendintas sudėtingas duomenų agregavimas iš trijų skirtingų išorinių šaltinių: „Alpha Vantage“ (rinkos duomenys), „Frankfurter“ (valiutų kursai) ir „NewsAPI“ (kontekstinė informacija). Sukurtas duomenų podėliavimo (**Caching**) mechanizmas vietinėje duomenų bazėje leido optimizuoti sistemos greitaveiką ir sumažinti priklausomybę nuo išorinių API kvotų.

4. **Kibernetinio saugumo užtikrinimas:**

- Įdiegta pilna autentifikacijos ir autorizacijos sistema naudojant **JWT (JSON Web Tokens)** standartą.
- Realizuotas **RBAC** (Role-Based Access Control) modelis, atskiriantis administratoriaus ir vartotojo teises.
- Vartotojų slaptažodžiai saugomi naudojant **BCrypt** maišos (hashing) algoritmą.
- API Gateway lygmenyje sukonfigūruotas užklausų ribojimas (**Rate Limiting**), apsaugantis sistemą nuo per didelės apkrovos ar DDoS atakų.

Apibendrinant, projektas leido praktiškai pritaikyti teorines žinias apie paskirstytas sistemas, API kūrimą ir saugumą. Sukurtas prototipas yra visiškai funkcionalus, geba apdoroti realius rinkos duomenis ir pateikti vertingas įžvalgas vartotojui, demonstratyviai parodant REST technologijų galimybes.